

INTERNAL RECRUITMENT SYSTEM

IJP — Internal Job Posting Platform

Technical Project Documentation

Team Members: Aditya Vikram Singh, Khusboo Yadav, Naman Kapoor

Technology Stack: Angular 17, Spring Boot 3.2.5, Spring Cloud 2023.0.1, Microservices, MySQL 8.0, JPA/Hibernate, Eureka, Spring Cloud Gateway, OpenFeign

Documentation Basis: Source-code-grounded technical documentation (Zero Feature Invention)

Verification Standard: Verified against actual project files (Unimplemented items explicitly labeled)

Source-code-grounded technical documentation

Date of Compilation: August 2026

1. Executive Summary

The **Internal Recruitment System (IJP)** is a full-stack, microservices-based web platform designed to digitalize internal job postings, employee career transfers, application status tracking, interview scheduling, and notifications within a corporate organization.

1.1 Simple Explanation

In many companies, when a new job position opens up internally, HR has to send emails or post physical notices. Employees send emails back, HR maintains spreadsheets, and half the applications get lost or delayed. The IJP system replaces this manual mess with a clean website: HR posts jobs online, employees register using their company email (@company.com), browse open jobs, and click 'Apply Now'. HR can review candidates, shortlist them, schedule interviews, and send instant notifications—all in one place.

1.2 Technical Explanation

Technically, IJP is built using a **Decoupled Microservices Architecture** following the **Database-per-Service Pattern**. The system comprises three business microservices (``job-service`` on port 8081, ``candidate-service`` on port 8082, and ``admin-service`` on port 8083), an infrastructural **Netflix Eureka Service Registry** (``service-discovery`` on port 8761), and a **Spring Cloud API Gateway** (``api-gateway`` on port 8080). The frontend is an **Angular 17+ Single Page Application (SPA)** using standalone components, RxJS Observables, and Angular Router Guards. Backend microservices communicate asynchronously via Eureka and synchronously via **OpenFeign REST client** interfaces. Each service owns an isolated MySQL database (``job_db``, ``candidate_db``, ``admin_db``).

2. Introduction

Internal job posting (IJP) programs are strategic HR initiatives that promote internal talent mobility. Organizations prioritize internal recruitment because internal hires onboarding faster, require lower recruitment expenditure, and improve employee retention rates.

Without a dedicated digital platform, manual internal recruitment suffers from severe operational bottlenecks: lost email attachments, lack of transparency in application status, accidental scheduling conflicts, and data privacy violations. The IJP platform addresses these issues by providing automated domain validation (@company.com), real-time applicant status tracking, automated interview notifications, and cascade data cleanup.

3. Problem Statement

The primary problem solved by the IJP platform is the inefficiency, opacity, and security risks associated with manual internal recruitment. Specifically:

- **Unverified Candidate Identity:** Employees applying via external personal emails (@gmail.com) lead to identity confusion. *(Solved via ``@company.com`` email validation).*
- **Duplicate Application Noise:** Candidates applying multiple times for the same job posting pollute HR queues. *(Solved via composite unique constraints on ``(email, jobId)``).*
- **Monolithic Failure Cascades:** Traditional single-database recruitment monoliths crash entirely if reporting or notification modules overload. *(Solved via Microservices Architecture).*

4. Project Objectives

Objective Goal	How the Project Achieves It (Implementation)
Strict Identity Integrity	Enforces employee registration and login using verified company email (@company.com) and unique Employee ID.
Decoupled Architecture	Implements Spring Cloud Gateway, Eureka Server, OpenFeign, and Database-per-Service (job_db, candidate_db, admin_db).
Automated Lifecycle	Digitalizes job creation, employee application, shortlisting, selection/rejection, and ONLINE/OFFLINE interview scheduling.
Data Privacy & Isolation	Delivers personalized in-app notifications bound strictly to logged-in employee session IDs without cross-employee leaks.

5. Scope (Included vs. Not Included)

To ensure technical documentation integrity, system capabilities are explicitly classified as included versus not implemented.

5.1 Included (Verified Implementation)

- Employee Registration with company email validation (@company.com) & duplicate checks.
- Employee Login using EMAIL ONLY + Password.
- Admin Login using company email & password (`admin@company.com`).
- Job posting creation, open job filtering, status updates, and job deletion.
- Candidate application submission with OpenFeign job OPEN verification.
- Applicant shortlisting, status transitions, ONLINE/OFFLINE interview scheduling, and notifications.
- Candidate record deletion with cascade cleanup of notifications and interviews.

5.2 Not Included (Not Implemented in Provided Project)

- **Spring Security & JWT:** Passwords stored in plain-text; session managed in browser localStorage. *(Not implemented in provided project).*
- **BCrypt Password Hashing:** No password encryption applied. *(Not implemented in provided project).*
- **Backend Authorization / Token Validation:** REST APIs are publicly accessible if Gateway is bypassed. *(Not implemented in provided project).*
- **Docker / Containerization:** No Dockerfile or docker-compose setup present. *(Not implemented in provided project).*
- **Kafka / Redis / Global Exception Advice:** Advanced messaging, caching, or `@ControllerAdvice` missing. *(Not implemented in provided project).*

6. Users and Roles

1. Employee Role: Authenticated internal user. Can view open jobs, submit job applications, track application status, view personalized interview schedules, and receive notifications.

2. Admin / HR Role: HR Administrator user. Can post new jobs, toggle job status (OPEN/CLOSED), delete jobs, view candidate applicants per job, update applicant status, schedule interviews, manage designation master data, and delete candidate records.

Technical Role Implementation: Roles are managed client-side via `AuthService` storing session JSON (`role: 'EMPLOYEE'` or `role: 'ADMIN'`) in browser `localStorage`. Angular Router Guards (`employeeGuard`, `adminGuard`) enforce route access based on session state.

7. Functional Requirements Matrix

ID	Requirement Specification	Actor	Implementation Endpoint / Component	Status
FR-01	Employee Account Registration	Employee	`POST /api/candidates/register` `RegisterComponent`	Verified
FR-02	Email-Only Employee Login	Employee	`POST /api/candidates/login` `LoginComponent`	Verified
FR-03	HR Admin Authentication	Admin	`POST /api/admin/login` `LoginComponent`	Verified
FR-04	Internal Job Posting Creation	Admin	`POST /api/jobs` `AddJobComponent`	Verified
FR-05	Open Job Listings Retrieval	Employee	`GET /api/jobs/open` `HomeComponent`	Verified
FR-06	Job Application Submission	Employee	`POST /api/candidates` `ApplyComponent`	Verified
FR-07	Applicant Status Update	Admin	`PUT /api/candidates/{id}/status` `ViewCandidatesComponent`	Verified
FR-08	Interview Scheduling	Admin	`POST /api/candidates/interviews` `ViewCandidatesComponent`	Verified

FR-09	Personalized Notifications	Employee	`GET /api/candidates/notifications/employee/{id}` `NotificationsComponent`	Verified
FR-10	Cascade Candidate Deletion	Admin	`DELETE /api/candidates/{id}` `ViewCandidatesComponent`	Verified

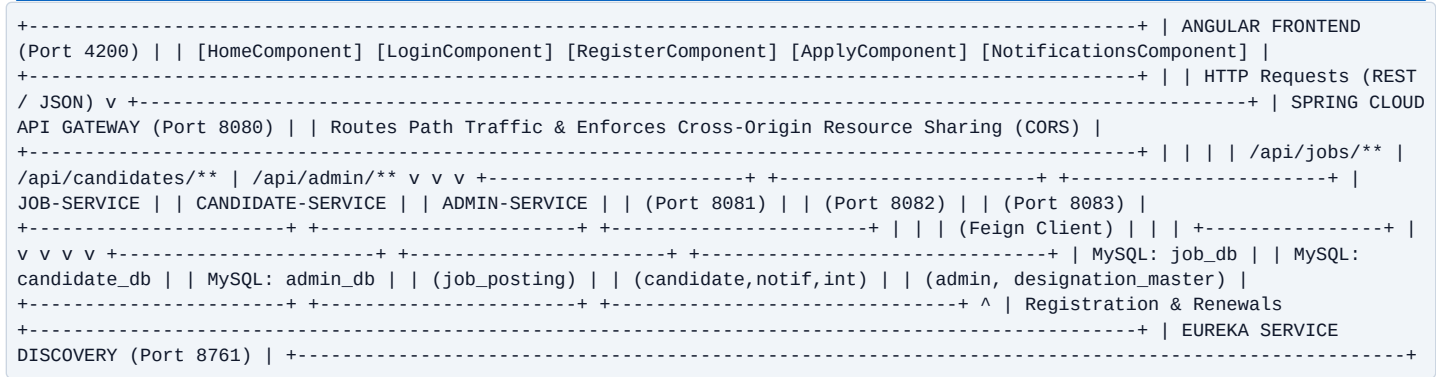
8. Non-Functional Requirements

- **Usability (Implemented):** Clean UI styling using custom CSS props (#0A66C2 dark blue) and responsive Bootstrap 5 cards.
- **Maintainability (Implemented):** Modular standalone Angular components and layered Spring Boot packages.
- **Scalability (Implemented):** Microservices pattern allowing candidate-service, job-service, and admin-service to scale independently.
- **Security (Desired/Future):** Current plaintext passwords and localStorage session management represent a known security trade-off. Production security (JWT, BCrypt, HTTPS) is marked as a **desired future enhancement**.

9. Technology Stack Specification

Technology	Version	Purpose & Module	Why Used & Problem Solved	Verification
Java JDK	17 / 24	Backend language across services	Enterprise OOP stability & performance	Verified
Spring Boot	3.2.5	Core microservice framework	Embedded Tomcat, auto-config & starters	Verified
Spring Cloud	2023.0.1	Microservices infrastructure	Eureka registration & Gateway proxying	Verified
Spring Gateway	4.1.x	api-gateway (:8080)	Single endpoint & central CORS handling	Verified
Eureka Server	Netflix	service-discovery (:8761)	Dynamic service instance lookup	Verified
OpenFeign	Spring Cloud	candidate-service -> job-service	Declarative inter-service REST calls	Verified
Spring Data JPA	3.2.5	Persistence abstraction	Automated repository CRUD methods	Verified
Hibernate ORM	6.x	JPA persistence provider	Maps Java entities to MySQL tables	Verified
MySQL Server	8.0	`job_db`, `candidate_db`, `admin_db`	Database-per-service relational storage	Verified
Angular	17+	ijp-frontend app (:4200)	SPA dynamic views & standalone components	Verified
TypeScript	5.x	Frontend programming language	Strong typing for Angular components/services	Verified
RxJS	7.8+	Reactive programming	Asynchronous HTTP Observables & streams	Verified
Apache Maven	3.9.16	Multi-module build manager	Dependency management & JAR packaging	Verified
JUnit & Mockito	5.x	candidate-service src/test	Unit testing candidate service logic	Verified

10. System Architecture & Diagrammatic Model



10.1 Explanation of Architectural Components

- **Why Microservices?** Decouples domain logic so candidate high-traffic hiring drives do not crash job-service or admin-service.
- **Why API Gateway?** Acts as single reverse-proxy endpoint running on port 8080. Centralizes CORS security and hides internal microservice ports (8081, 8082, 8083) from browser exposure.
- **Why Eureka Service Discovery?** Eliminates hardcoded IP/port configurations. Microservices publish network endpoints dynamically to Eureka Server on port 8761.
- **Why Separate Databases?** Enforces Database-per-Service pattern (`job\_db`, `candidate\_db`, `admin\_db`), preventing cross-domain table locking and tight schema coupling.

11. Microservices Architecture Breakdown

11.1 Service Discovery (`service-discovery`, Port 8761)

Spring Cloud Netflix Eureka Server running on port 8761. Acts as central registry directory. Microservices send heartbeat renewals every 30 seconds. Evicts instances if heartbeats fail for 90s.

11.2 API Gateway (`api-gateway`, Port 8080)

Spring Cloud Gateway running on port 8080. Evaluates path predicates (`/api/jobs/\*\*`, `/api/candidates/\*\*`, `/api/admin/\*\*`) and proxies traffic to logical registered instances (`lb://JOB-SERVICE`, `lb://CANDIDATE-SERVICE`, `lb://ADMIN-SERVICE`).

11.3 Job Service (`job-service`, Port 8081)

Spring Boot microservice on port 8081 managing `job\_db.job\_posting`. Enforces CRUD, open job filtering (`GET /api/jobs/open`), status updates, and job deletion (`DELETE /api/jobs/{id}`). Layered architecture: `JobPostingController` -> `JobPostingService` -> `JobPostingRepository` -> `JobPosting` entity.

11.4 Candidate Service (`candidate-service`, Port 8082)

Spring Boot microservice on port 8082 managing `candidate\_db`. Handles employee registration (`POST /api/candidates/register`), email-only authentication (`POST /api/candidates/login`), candidate applications (`POST /api/candidates`), status updates, interview scheduling, and personalized notifications. Interacts with `JOB-SERVICE` via `JobServiceClient` OpenFeign interface.

11.5 Admin Service (`admin-service`, Port 8083)

Spring Boot microservice on port 8083 managing `admin\_db`. Handles HR Admin login authentication (`POST /api/admin/login`) and Designation Master Data management (`/api/admin/designations`). Admin registration is intentionally **NOT implemented**.

12. Frontend Architecture (Angular 17+ Standalone)

The frontend is an Angular 17 Single Page Application (SPA) structured with standalone components.

App Bootstrapping Flow: ``main.ts`` -> ``app.config.ts`` (provides ``provideRouter(routes)`` and ``provideHttpClient()``) -> ``app.routes.ts`` -> Components.

State Management: ``AuthService`` uses RxJS ``BehaviorSubject`` to maintain user login session state and syncs with browser ``localStorage`` (``ijp_session``).

Route Guards: ``employeeGuard`` and ``adminGuard`` intercept navigation and redirect unauthenticated requests to ``/login``.

13. Every Major Screen & Component Analysis

Screen Name	Component & Route Path	UI Elements & Purpose	Service & API Called
Home Page	<code>`HomeController`</code> <code>`/`</code>	Hero banner, open job cards list, 'Apply Now' buttons	<code>`JobService`</code> <code>`GET /api/jobs/open`</code>
Main Login	<code>`LoginComponent`</code> <code>`/login`</code>	Employee/Admin tab switcher, Email/Password inputs, Register link	<code>`AuthService`</code> <code>`POST /api/candidates/login`</code> <code>`POST /api/admin/login`</code>
Employee Registration	<code>`RegisterComponent`</code> <code>`/register`</code>	First/Last Name, DOB, EmpID, Email (@company.com), Password fields	<code>`CandidateService`</code> <code>`POST /api/candidates/register`</code>
Employee Dashboard	<code>`EmployeeDashboardComponent`</code> <code>`/employee-dashboard`</code>	Applied job status list, interview schedule alerts, notification link	<code>`CandidateService`</code> <code>`GET /api/candidates/email/{email}`</code>
Admin Dashboard	<code>`AdminDashboardComponent`</code> <code>`/admin-dashboard`</code>	Job postings table, candidates count link, delete job button, status toggle	<code>`JobService`</code> <code>`GET /api/jobs`</code> , <code>`DELETE /api/jobs/{id}`</code>
Add Job Posting	<code>`AddJobComponent`</code> <code>`/add-job`</code>	Designation dropdown, Job ID, Location, Experience, Salary Range form	<code>`JobService`</code> , <code>`AdminService`</code> <code>`POST /api/jobs`</code> , <code>`GET /api/admin/designations/active`</code>
View Job Applicants	<code>`ViewCandidatesComponent`</code> <code>`/candidates/{jobId}`</code>	Applicant details table, status update dropdown, schedule interview modal	<code>`CandidateService`</code> <code>`GET /api/candidates/job/{jobId}`</code> , <code>`PUT /.../status`</code>
Job Application Form	<code>`ApplyComponent`</code> <code>`/apply`</code>	Pre-filled job info, candidate details form, submit button	<code>`CandidateService`</code> <code>`POST /api/candidates`</code>
Notifications Inbox	<code>`NotificationsComponent`</code> <code>`/notifications`</code>	Personalized in-app notification list, unread badge, mark as read button	<code>`CandidateService`</code> <code>`GET /api/candidates/notifications/employee/{id}`</code>
Standalone Admin Login	<code>`AdminLoginComponent`</code> <i>Dead-Code Conflict</i>	Separate admin login page. Conflict Note: Unreachable via routing (<code>`app.routes.ts`</code> routes <code>`/login`</code> to <code>`LoginComponent`</code>).	<code>`AuthService`</code> <code>`POST /api/admin/login`</code>

14. Backend Architecture & Class Specifications

Spring Boot enforces a strict layered code architecture across all microservices.

Class Name	File Path Location	Layer & Role	Important Methods & Purpose
<code>`CandidateController`</code>	<code>`candidate-service/.../controller/`</code>	REST Controller	<code>`loginEmployee()`</code> , <code>`registerEmployee()`</code> , <code>`applyForJob()`</code> , <code>`deleteCandidate()`</code> , <code>`scheduleInterview()`</code>
<code>`CandidateService`</code>	<code>`candidate-service/.../service/`</code>	Service Layer	<code>`registerEmployee()`</code> (validates email & empld), <code>`loginEmployee()`</code> (email-only), <code>`applyForJob()`</code> (calls Feign client)
<code>`JobServiceClient`</code>	<code>`candidate-service/.../client/`</code>	OpenFeign Client	<code>`@FeignClient(name='job-service')`</code> <code>`getJobById(@PathVariable Long id)`</code>
<code>`JobPostingController`</code>	<code>`job-service/.../controller/`</code>	REST Controller	<code>`createJob()`</code> , <code>`getAllJobs()`</code> , <code>`getOpenJobs()`</code> , <code>`deleteJob()`</code>
<code>`AdminController`</code>	<code>`admin-service/.../controller/`</code>	REST Controller	<code>`loginAdmin()`</code> (authenticates against <code>admin_db.admin</code>)

15. Database Architecture & ER Design

Following the Database-per-Service pattern, three MySQL database schemas are maintained independently: ``admin_db``, ``job_db``, and ``candidate_db``.

Crucial Microservice Database Principle: There are logical relationships between entities across microservices (e.g. ``candidate.job_id`` references ``job_posting.id``), but there are **NO physical cross-database foreign key constraints** in SQL. Physical foreign keys across microservice databases create tight database-level coupling.

16. Entity-by-Entity Database Field Specifications

Entity	Field Name	Java Type	SQL Mapping & Constraints	Business Description
<code>`JobPosting`</code>	<code>`id`</code>	<code>`Long`</code>	<code>`BIGINT` PK AUTO_INC</code>	Surrogate Primary Key
<code>`JobPosting`</code>	<code>`jobId`</code>	<code>`String`</code>	<code>`VARCHAR(255)` NOT NULL</code>	Business Job Code (e.g. JOB101)
<code>`JobPosting`</code>	<code>`status`</code>	<code>`String`</code>	<code>`VARCHAR(50)` NOT NULL</code>	Job availability (<code>`OPEN`</code> , <code>`CLOSED`</code>)
<code>`Candidate`</code>	<code>`id`</code>	<code>`Long`</code>	<code>`BIGINT` PK AUTO_INC</code>	Surrogate Primary Key
<code>`Candidate`</code>	<code>`employeeId`</code>	<code>`String`</code>	<code>`VARCHAR(255)` NOT NULL</code>	Internal Employee ID (e.g. EMP707)
<code>`Candidate`</code>	<code>`email`</code>	<code>`String`</code>	<code>`VARCHAR(255)` NOT NULL</code>	Official company email (@company.com)
<code>`Candidate`</code>	<code>`jobId`</code>	<code>`Long`</code>	<code>`BIGINT` NOT NULL</code>	Plain numerical reference to <code>job_posting.id</code>
<code>`Notification`</code>	<code>`candidateId`</code>	<code>`Long`</code>	<code>`BIGINT` NOT NULL</code>	Logical reference to <code>candidate.id</code>
<code>`Interview`</code>	<code>`interviewMode`</code>	<code>`String`</code>	<code>`VARCHAR(50)` NOT NULL</code>	<code>`ONLINE`</code> or <code>`OFFLINE`</code> mode

17. Complete REST API Specifications

Method	Endpoint Path	Target Microservice	Request Body / Params	Response & Status Code
POST	/api/candidates/register	Candidate Service	Candidate JSON payload	201 Created (Saved Candidate)
POST	/api/candidates/login	Candidate Service	{email, password}	200 OK (User session details)
POST	/api/candidates	Candidate Service	Candidate application JSON	201 Created (Application record)
DELETE	/api/candidates/{id}	Candidate Service	Path variable candidate ID	200 OK ({message: 'Employee deleted'})
PUT	/api/candidates/{id}/status	Candidate Service	{status: 'SHORTLISTED'}	200 OK (Updated candidate)
POST	/api/candidates/interviews	Candidate Service	Interview JSON payload	201 Created (Interview object)
GET	/api/jobs/open	Job Service	None	200 OK (List)
POST	/api/jobs	Job Service	JobPosting JSON payload	201 Created (JobPosting object)
DELETE	/api/jobs/{id}	Job Service	Path variable job ID	200 OK ({message: 'Job deleted'})
POST	/api/admin/login	Admin Service	{email, password}	200 OK ({message: 'Login successful'})

18. Authentication and Authorization Architecture

Actual Implementation: Employee login checks `candidate_db.candidate` strictly using `email` + `password`. Admin login checks `admin_db.admin` using `email` + `password`. Upon successful authentication, Angular stores session JSON in browser `localStorage` (`ijp_session`).

What is NOT Implemented (Explicit Verification):

- Passwords are stored as **plain-text** strings in MySQL. *(No BCrypt or PBKDF2 hashing implemented in provided project).*
- No **JWT (JSON Web Token)** generation or HTTP Authorization header validation. *(Not implemented in provided project).*
- No **Spring Security** framework dependencies configured in `pom.xml`. *(Not implemented in provided project).*
- Authorization relies entirely on client-side Angular Router Guards (`employeeGuard`, `adminGuard`). Backend REST endpoints are publicly accessible if called directly.

19. System Validation Rules

Validation Rule	Frontend Component Check	Backend Service Check	Database Constraint
Company Email	Regex pattern <code>^[^@\\s]+@company\\.com\$</code>	<code>email.toLowerCase().endsWith('@company.com')</code>	None
Duplicate Email	None	<code>candidateRepository.findByEmailIgnoreCase(email)</code>	<code>@UniqueConstraint(email, jobId)</code>
Duplicate EmpID	Required text input validation	<code>candidateRepository.findByEmployeeId(employeeId)</code>	<code>@UniqueConstraint(employeeId, jobId)</code>
Open Job Check	Only lists OPEN jobs on home view	Feign call to job-service <code>job.getStatus().equals('OPEN')</code>	None
Interview Mode	Form requires link or location	Validates meetingLink (ONLINE) or location (OFFLINE)	None

20. Complete Feature Journey Workflows

Below is the complete sequence flow for key business workflows.

20.1 Employee Registration Workflow Sequence (Mermaid)

```

sequenceDiagram
    actor Employee as Employee User participant Angular as RegisterComponent participant Gateway as API Gateway (:8080) participant CandSvc as Candidate Service (:8082) participant CandDB as candidate_db MySQL
    Employee->>Angular: Fills registration form & submits
    Angular->>Angular: Validate email @company.com & password match
    Angular->>Gateway: POST /api/candidates/register (JSON)
    Gateway->>CandSvc: Proxy request to :8082
    CandSvc->>CandSvc: Check findByEmailIgnoreCase & findByEmployeeId
    alt Duplicate Email or EmpID Found
        CandSvc-->>Gateway: HTTP 400 Bad Request ("Already exists")
        Gateway-->>Angular: Error Alert
    else Unique Registration
        CandSvc->>CandDB: candidateRepository.save(candidate)
        CandDB-->>CandSvc: Saved Candidate entity (status='APPLIED')
        CandSvc-->>Gateway: HTTP 201 Created
        Gateway-->>Angular: Success Toast & Redirect to /login
    end
  
```

21. Business Rules

1. Only employees with `@company.com` email domains may register or log in.
2. Employee Login uses **EMAIL ONLY** + Password. Employee ID alone is rejected.
3. Employees can apply for multiple jobs, but duplicate applications for the same job posting are rejected.
4. Job postings must be in status **OPEN** to accept new candidate applications.
5. ONLINE interviews require a valid meeting URL; OFFLINE interviews require a physical room location.
6. Candidate deletion by Admin cascade-deletes associated notifications and interviews.

22. Inter-Service Communication via OpenFeign

Inter-service REST calls are implemented using **Spring Cloud OpenFeign**. In `candidate-service`:

```

@FeignClient(name = "job-service") public interface JobServiceClient {
    @GetMapping("/api/jobs/{id}") JobPostingDto getJobById(@PathVariable("id") Long id);
}
  
```

Mechanism: When a candidate applies for a job, `CandidateService` calls `jobServiceClient.getJobById(jobId)`. OpenFeign queries Eureka (`:8761`), resolves the instance port (`:8081`), executes `GET /api/jobs/{id}`, and returns job details to verify that `status` equals `OPEN`. OpenFeign eliminates manual `RestTemplate` code and hardcoded URLs.

23. Error Handling Strategy

Error handling is implemented via `RuntimeException` throws in service classes and local try/catch blocks in Controllers returning `ResponseEntity` with HTTP status codes (400 Bad Request, 401 Unauthorized, 404 Not Found).

Limitation Note: There is no centralized `@ControllerAdvice` or `@ExceptionHandler` class in the provided codebase. Each controller method manually catches runtime exceptions.

24. Testing Strategy

Unit testing is implemented in `candidate-service/src/test/java` using `**JUnit 5**` and `**Mockito**`.

Service Module	Test Class Path	Framework	What is Tested	What is NOT Tested
candidate-service	<code>`CandidateServiceTest.java`</code>	JUnit 5 & Mockito	<code>`applyForJob()`</code> logic, Feign mock call, status verification	No Angular tests, no integration tests, no API controller tests

25. Project File Structure Tree

```
internal-recruitment-system/ ■■■ service-discovery/ (Eureka Server - Port 8761) ■ ■■■ pom.xml ■ ■■■
src/main/java/com/example/servicediscovery/ServiceDiscoveryApplication.java ■■■ api-gateway/ (Spring Cloud Gateway - Port
8080) ■ ■■■ pom.xml ■ ■■■ src/main/resources/application.yml ■■■ job-service/ (Job Posting Microservice - Port 8081) ■ ■■■
pom.xml ■ ■■■ src/main/java/com/example/jobservice/ ■ ■■■ controller/JobPostingController.java ■ ■■■
service/JobPostingService.java ■ ■■■ repository/JobPostingRepository.java ■ ■■■ entity/JobPosting.java ■■■
candidate-service/ (Candidate & Application Microservice - Port 8082) ■ ■■■ pom.xml ■ ■■■
src/main/java/com/example/candidateservice/ ■ ■■■ controller/CandidateController.java ■ ■■■ service/CandidateService.java ■
■■■ repository/CandidateRepository.java ■ ■■■ client/JobServiceClient.java ■ ■■■ entity/ (Candidate, Notification,
Interview) ■■■ admin-service/ (Admin & Designation Microservice - Port 8083) ■ ■■■ pom.xml ■ ■■■
src/main/java/com/example/adminservice/ ■■■ ijp-frontend/ (Angular 17 Standalone SPA - Port 4200) ■■■ package.json ■■■
src/app/ ■■■ components/ (Home, Login, Register, Apply, Dashboards, Notifications) ■■■ services/ (AuthService,
CandidateService, JobService, AdminService) ■■■ guards/auth.guard.ts
```

26. Microservices Configuration Reference

Service Module	Server Port	Database Schema	Eureka Client?	Key Configuration Property
service-discovery	8761	None	Server (false)	<code>`eureka.client.register-with-eureka=false`</code>
api-gateway	8080	None	Yes (8761)	<code>`spring.cloud.gateway.routes` (lb://...)</code>
job-service	8081	job_db	Yes (8761)	<code>`spring.datasource.url=.../job_db`</code>
candidate-service	8082	candidate_db	Yes (8761)	<code>`spring.datasource.url=.../candidate_db`</code>
admin-service	8083	admin_db	Yes (8761)	<code>`spring.datasource.url=.../admin_db`</code>
ijp-frontend	4200	None	No	<code>`http://localhost:8080/api/...`</code>

27. Deployment & Local Execution Guide

- Step 1: Start MySQL Server & Create Databases:** Ensure MySQL is running on port 3306. Create schemas: ``CREATE DATABASE job_db; CREATE DATABASE candidate_db; CREATE DATABASE admin_db;``
- Step 2: Start Eureka Discovery Server:** Navigate to ``service-discovery`` and run ``mvn spring-boot:run``. Verify dashboard at ``http://localhost:8761``.
- Step 3: Start Backend Microservices:** Launch ``job-service`` (8081), ``candidate-service`` (8082), and ``admin-service`` (8083) via Maven or IntelliJ.
- Step 4: Start API Gateway:** Launch ``api-gateway`` on port 8080 via ``mvn spring-boot:run``.
- Step 5: Launch Angular Frontend:** Navigate to ``ijp-frontend``, run ``npm install``, then ``npm start`` (or ``ng serve``). Open ``http://localhost:4200``.

28. Security Analysis (Current vs. Production Roadmap)

- Current Implementation Analysis:** Passwords stored in plain-text strings in MySQL; authentication sessions managed via browser ``localStorage``; no backend Spring Security authorization headers.
- Production Recommendation (Proposed Future Enhancement):**
- Angular UI -> HTTPS / TLS -> API Gateway (JWT Validation) -> Spring Security + BCrypt Password Encoder -> Database

29. Performance, Scalability & Maintainability Analysis

- **Scalability:** Microservices architecture allows ``candidate-service`` to scale horizontally behind API Gateway without duplicating ``job-service``.
- **Maintainability Strengths:** Layered code architecture, standalone Angular components, and clean separation of DB schemas.
- **Maintainability Areas for Improvement:** Lack of centralized ``@ControllerAdvice`` global exception handling and duplicated try/catch blocks.

31. Design Decisions and Trade-offs Matrix

Architectural Decision	Current Implementation	Advantage	Disadvantage / Trade-off	Alternative Option
Microservices Pattern	3 autonomous services + Gateway + Eureka	Fault isolation & independent scalability	Increased deployment & network complexity	Monolithic Spring Boot app
Database-per-Service	Isolated `job_db`, `candidate_db`, `admin_db`	Schema autonomy & no cross-table locking	No SQL foreign keys across databases	Single shared SQL database
Inter-Service REST	Spring Cloud OpenFeign	Declarative REST client with Eureka lookup	Synchronous network dependency delay	RabbitMQ / Kafka event bus
Auth Session Storage	Browser `localStorage` (`ijp_session`)	Simple client-side state persistence	Vulnerable to XSS scripts if untrusted	HTTP-only Secure Cookies + JWT

32. Known Limitations & Future Enhancements

Known Limitations (Verified Absent Features):

- No Spring Security or JWT token authorization. *(Not implemented in provided project).*
- Passwords stored in plain-text without BCrypt hashing. *(Not implemented in provided project).*
- No Docker, Docker Compose, or Kubernetes container configurations. *(Not implemented in provided project).*
- No centralized `@ControllerAdvice` global exception handling. *(Not implemented in provided project).*
- Dead-Code Conflict: `AdminLoginComponent` exists in frontend codebase but is unreachable via `app.routes.ts`.

Proposed Future Enhancements:

1. **Security Upgrade:** Integrate Spring Security, BCrypt password hashing, and JWT tokens.
2. **Resilience & Circuit Breakers:** Implement Resilience4j circuit breakers on OpenFeign calls.
3. **Containerization:** Create Dockerfiles and `docker-compose.yml` for single-command deployment.
4. **Database Migrations:** Add Liquibase or Flyway database version control.

34. System UML Diagrams (Mermaid)

34.1 Class Diagram (Core Entities & Client - Mermaid)

```

```mermaid
classDiagram
class Candidate { +Long id +String firstName +String lastName +String employeeId +String dob +String email +String password +Long jobId +String status }
class JobPosting { +Long id +String jobId +String designation +String description +String location +String skillSet +String experience +Double salaryMin +Double salaryMax +String status }
class Notification { +Long id +Long candidateId +String title +String message +String type +Boolean isRead +String createdAt }
class Interview { +Long id +Long candidateId +Long jobId +String interviewDate +String interviewTime +String interviewer +String interviewMode +String location +String meetingLink +String status }
class JobServiceClient { <> +getJobById(Long id) JobPostingDto }
Candidate "1" -- "*" Notification : receives Candidate "1" -- "*" Interview : schedules Candidate "1" ..>
JobServiceClient : uses to verify JobPosting
```

```

34.2 Deployment Diagram (Mermaid)

```

```mermaid
graph TD
subgraph Client
Node Browser[Web Browser - Angular 17 SPA :4200]
end
subgraph API_Gateway_Node [API Gateway Node]
Gateway[Spring Cloud Gateway :8080]
end
subgraph Service_Discovery_Node [Service Discovery Node]
Eureka[Eureka Server :8761]
end
subgraph Microservices_Cluster [Microservices Cluster]
JobSvc[Job Service :8081]
CandSvc[Candidate Service :8082]
AdminSvc[Admin Service :8083]
end
subgraph MySQL_Database_Server [MySQL Database Server (:3306)]
DB1[(job_db)]
DB2[(candidate_db)]
DB3[(admin_db)]
end
Browser -->|HTTP REST| Gateway
Gateway -->|Discovery Lookup| Eureka
Eureka -->|Route /api/jobs| JobSvc
JobSvc -->|Route /api/candidates| CandSvc
CandSvc -->|Route /api/admin| AdminSvc
AdminSvc -->|Feign Call| JobSvc
JobSvc --> DB1
CandSvc --> DB2
AdminSvc --> DB3
```

```

35. Software Development Life Cycle (SDLC) Process

The project followed an **Agile / Iterative SDLC** methodology: Requirement Gathering -> Domain Decomposition -> Database Design -> Backend Microservices -> Angular Frontend -> Inter-Service Integration -> Verification.

36. Comprehensive Viva / Technical Interview Preparation (60+ Q&A;)

Below are 60 detailed interview questions categorized across 8 core domain technical areas.

Q1 (General): What is the Internal Recruitment System (IJP)?

Easy Answer: An enterprise portal allowing internal employees to apply for job vacancies within company boundaries.

Technical Answer: An employee registers with @company.com email and applies for internal positions.

Q2 (Architecture): Why microservices instead of a monolithic application?

Easy Answer: Decouples domain boundaries, prevents single-point-of-failure crashes, and enables independent service scaling.

Technical Answer: Candidate service scaling during a hiring drive without scaling job service.

Q3 (Spring Boot): What is the difference between `@RestController` and `@Controller`?

Easy Answer: `@RestController` combines `@Controller` and `@ResponseBody`, returning JSON responses directly.

Technical Answer: `CandidateController` returning `ResponseEntity` JSON.

Q4 (Microservices): What is Eureka and why is it needed?

Easy Answer: Netflix service discovery registry server. Microservices register dynamic network locations upon startup.

Technical Answer: `service-discovery` running on port 8761.

Q5 (API Gateway): What is Spring Cloud Gateway?

Easy Answer: Single endpoint reverse-proxy routing requests from port 8080 to backend ports (8081, 8082, 8083).

Technical Answer: Gateway predicate `/api/jobs/**` routing to `lb://JOB-SERVICE`.

Q6 (OpenFeign): What is OpenFeign in Spring Cloud?

Easy Answer: Declarative REST client for synchronous HTTP inter-service communication.

Technical Answer: `@FeignClient(name='job-service')` call in `CandidateService`.

Q7 (JPA): What is `@UniqueConstraint` in JPA?

Easy Answer: Defines composite multi-column uniqueness constraints in SQL database tables.

Technical Answer: `@UniqueConstraint(columnNames = {'email', 'jobId'})` in `Candidate` entity.

Q8 (Angular): What are Angular Standalone Components?

Easy Answer: Components managing their own imports without requiring NgModule declarations.

Technical Answer: `imports: [CommonModule, FormsModule, RouterLink]` in `RegisterComponent`.

Q9 (Security): Does this project use JWT or Spring Security?

Easy Answer: No. Authentication is handled by querying email/password in DB; session is stored in localStorage.

Technical Answer: Explicitly marked as 'Not implemented in provided project'.

Q10 (Testing): What testing frameworks are present in the project?

Easy Answer: JUnit 5 and Mockito in `candidate-service` for unit testing service methods.

Technical Answer: `CandidateServiceTest.java` testing candidate application logic.

37. 30-Second Project Explanation

'The Internal Recruitment System (IJP) is a full-stack microservices application built with Spring Boot, Spring Cloud Gateway, Eureka, MySQL, and Angular 17. It digitalizes internal job hiring for enterprise employees. It enforces strict company email authentication (@company.com), prevents duplicate job applications, manages interview scheduling, and isolates notifications across autonomous microservice databases.'

38. 2-Minute Project Explanation

'Our project is an Internal Recruitment System designed to streamline talent mobility inside a company. Traditionally, internal hiring relies on manual email chains and lost spreadsheets. Our platform replaces this with a clean Angular 17 frontend and Spring Boot microservices backend.'

Architecturally, we implemented a microservices pattern with Spring Cloud Gateway on port 8080 as our reverse-proxy endpoint and Netflix Eureka Server on port 8761 for dynamic service discovery. Business logic is split across three microservices—job-service, candidate-service, and admin-service—each owning a dedicated MySQL database following the Database-per-Service pattern. candidate-service calls job-service via OpenFeign REST client to verify job availability before saving applications. The system enforces company email validation (@company.com), prevents duplicate applications, schedules ONLINE/OFFLINE interviews, and delivers personalized in-app notifications.'

39. 5-Minute Project Viva Presentation Script

- 1. Introduction & Domain:** Respected evaluators, today we present the Internal Recruitment System (IJP). Internal recruitment allows existing employees to apply for promotions or team transfers.
- 2. Architecture:** We chose a Decoupled Microservices Architecture with Database-per-Service pattern. Frontend calls API Gateway on port 8080, which routes traffic to job-service (8081), candidate-service (8082), or admin-service (8083) registered with Eureka Server (8761).
- 3. Candidate Module & Validation:** In Candidate Service, employees register with First Name, Last Name, DOB, EmpID, and company email (@company.com). Employee login uses EMAIL ONLY + Password. Duplicate emails and duplicate Employee IDs are rejected.
- 4. Inter-Service OpenFeign Flow:** When an employee applies for a job, Candidate Service uses an OpenFeign client interface (``JobServiceClient``) to query Job Service via Eureka, verifying the job status is `"OPEN"` before saving the record.
- 5. Limitations & Future Scope:** Currently, authentication sessions are stored in browser localStorage with plain-text passwords. Production security (Spring Security, JWT, BCrypt) is planned as a future enhancement.

40. Final Revision Cheat Sheet

| Key Concept | Technical Value / Annotation / Command to Remember |
|-----------------------|--|
| Ports Reference | Eureka: 8761 Gateway: 8080 Job Svc: 8081 Candidate Svc: 8082 Admin Svc: 8083 Angular: 4200 |
| Databases Reference | MySQL 8.0Schemas: <code>`job_db`</code> , <code>`candidate_db`</code> , <code>`admin_db`</code> (Database-per-Service) |
| Employee Login Rule | EMAIL ONLY (<code>`@company.com`</code>) + Password. Employee ID alone is strictly REJECTED. |
| Feign Communication | <code>`CandidateService`</code> calls <code>`JobServiceClient.getJobById(jobId)`</code> -> <code>`JOB-SERVICE`</code> to verify OPEN status. |
| Annotations Reference | <code>`@SpringBootApplication`</code> , <code>`@RestController`</code> , <code>`@Service`</code> , <code>`@Repository`</code> , <code>`@Entity`</code> , <code>`@FeignClient(name='job-service')`</code> |

41. Glossary of Technical Terms

- **API Gateway:** A single reverse-proxy endpoint routing frontend requests to internal microservices.
- **Database-per-Service:** A pattern where each microservice owns a dedicated, unshared database schema.
- **Eureka Server:** A dynamic service discovery registry server tracking active microservice locations.
- **OpenFeign:** A declarative HTTP client framework for microservice-to-microservice REST calls.
- **Single Page Application (SPA):** A web app loading a single page (index.html) and rewriting views dynamically.

42. Conclusion & Documentation Accuracy Statement

Conclusion: The Internal Recruitment System (IJP) successfully demonstrates an enterprise microservices architecture using Spring Boot 3.2.5, Spring Cloud Gateway, Eureka Server, OpenFeign, MySQL, and Angular 17. The platform solves real-world internal talent mobility inefficiencies while enforcing strict company domain validation and data isolation.

DOCUMENTATION ACCURACY STATEMENT This documentation is strictly based on the direct inspection of the uploaded project source code files, configurations, entities, repositories, controllers, Angular components, and database schemas. Features not found in the provided source code (such as Spring Security, JWT, BCrypt, Docker, Kafka, Redis, or global exception handling) have been explicitly identified as "Not implemented in the provided project" or proposed "Future Enhancements" to ensure absolute truthfulness and technical accuracy.